PROJECT REPORT

AI-Assisted Web Application Project

Semester 3 – CSE (AI/ML)

Project Title

Lecture Summarizer — AI-Powered Study Assistant

Submitted by

Student Name: Lakkireddy Mokshith Manikanta Reddy

Enrollment No.: _____

Division: _____

Contents

1. Introduction	3
2. Target User	3
3. Tools Used	3
4. Prompt Engineering Process	3
4.1 Idea / Problem Analysis	3
4.2 Requirements / Features	3
4.3 UI / Layout Design	3
4.4 Code Generation	3
4.5 Testing	3
4.6 Debugging and Improvement	3
5. Application Design and Development	4
5.1 System Architecture Flow	4
5.2 UI Components & Page Structure	4
6. Final Application	4
7. Conclusion	4

1. Introduction

During exam preparation and internal assessments, college students face the challenge of reviewing 40 to 60 dense slide decks per lecture. Reading through raw slides takes hours, and during fast-paced revision, critical definitions and exam-relevant concepts are easily missed. Standard text summarizers generate unstructured paragraphs without categorizing insights into a format suitable for active study.

To address this issue, I developed **Lecture Summarizer**, a Flask web application that takes either pasted notes or uploaded PDF slides and converts them into a 4-part structured study guide: Key Concepts, Definitions, Summary, and Exam Questions. Prompt engineering and AI assistance were utilized across all phases, including requirement scoping, UI styling, and multi-model failover debugging.

2. Target User

The primary users are **undergraduate university students** (especially in engineering and sciences) requiring rapid, structured revision before tests. Secondary users include **Teaching Assistants and lecturers** who need to prepare quick recap handouts and sample quiz questions from syllabus materials without manual formatting.

3. Tools Used

• **Stack:** Python 3.14, Flask 3.1 (backend web server), PyPDF2 3.0 (PDF text extraction), python-dotenv (environment variables), HTML5/CSS3 (responsive card layout with Inter font).
• **AI Inference:** Google Gemini API via google-genai SDK, with a multi-model fallback engine (gemini-3.5-flash-lite, gemini-flash-lite-latest, gemini-3.8-flash).
• **AI-Assisted Tools:** Antigravity Agent tools for prompt design, code scaffolding, automated testing, and live debugging.

4. Prompt Engineering Process

The application was built iteratively using targeted prompt engineering at each stage:

Phase & Purpose	Engineered Prompt & Implementation Strategy
4.1 Idea / Problem Analysis *Purpose: Brainstorm realistic, high-impact college project concepts.*	*"Give me simple ideas for a project that is useful to a university student. Suggest options across study productivity and campus life, and point out which one is best suited for an AI web app demo."*
4.2 Requirements & Features *Purpose: Define functional architecture and output format constraints.*	*"I want to build a Lecture Summarizer web app using Python Flask and Google Gemini API. It should accept pasted text or uploaded PDF files and return four specific sections: Key Concepts, Definitions, Summary, and Exam Questions. Also include session logging."*
4.3 UI / Layout Design *Purpose: Create a distraction-free, tabbed academic interface.*	*"Write an HTML/CSS UI with an academic blue theme. Use a two-tab interface on the home page: Tab 1 for typing/pasting text, Tab 2 for uploading a PDF. Render results in 4 separate visual cards with a session metadata strip showing execution latency."*
4.4 Code Generation *Purpose: Enforce rigid markdown delimiters for deterministic regex parsing.*	*"Write summarizer.py using google-genai SDK. Instruct the model to strictly output exact headers: '## Key Concepts', '## Important Definitions', '## Summary Paragraph', and '## Possible Exam Questions'. Write a parser that splits the response into a structured dict."*
4.5 Testing *Purpose: Programmatically verify endpoint status codes and session logs.*	*"Write a Python test script using Flask test_client that sends sample operating systems lecture notes to POST / and asserts that all four study sections are returned with an HTTP 200 status code."*
4.6 Debugging & Improvement *Purpose: Overcome transient 503 errors and sanitize bracketed terms.*	*"The Gemini free tier occasionally returns 503 UNAVAILABLE on gemini-3.8-flash due to traffic spikes. Update summarizer.py to automatically fallback through candidate models: gemini-3.5-flash-lite, gemini-flash-lite-latest, and gemini-3.8-flash. Also strip square brackets from terms."*

5. Application Design and Development

The application employs a lightweight modular structure with clean separation of concerns between routing (app.py), PDF buffer processing (PyPDF2), AI failover summarization (summarizer.py), and session logging (logger.py).

```
System Architecture & Data Flow:
User (Browser) ➔ [Paste Text / Upload PDF] ➔ PyPDF2 Text Extraction
➔ Gemini Multi-Model Failover Engine [gemini-3.5-flash-lite / flash-lite-latest / 3.8-flash]
➔ Deterministic Regex Section Parser ➔ Render 4 Study Cards (summary.html)
➔ Session Telemetry Logger (logs/sessions.json)
```

UI Components & Navigation Flow:

• **Home Page (/):** Academic header with brand navigation, dual-tab card allowing users to switch between pasting raw notes and uploading PDF slides.

• **Summary Results Page (/ POST):** Renders a metadata strip (Session ID, characters analyzed, inference time in seconds) and four distinct visual cards: *Key Concepts* (bullet points), *Important Definitions* (term and definition pairs), *Summary Paragraph* (executive overview), and *Possible Exam Questions* (numbered questions for self-assessment). Includes an expandable raw response toggle.

• **Session History Page (/logs):** Tabular audit log displaying timestamped records of past summarizations with character counts and processing latencies.

6. Final Application

The completed application is running locally and verified through automated end-to-end tests. Across sample test cases (including Operating Systems virtual memory notes and Computer Networks TCP/UDP slides), the system achieved an average summarization latency of 4 to 8 seconds with zero unhandled exceptions.

• **Key Working Features:** Robust PDF text extraction, resilient multi-model failover ensuring uptime during API demand spikes, deterministic structured parsing, and persistent JSON telemetry.

• **GitHub Repository Link:** https://github.com/lakkireddymokshithmanikantareddy/lecture-summarizer

7. Conclusion

Project Summary: This project demonstrated the successful end-to-end development of an AI-assisted web utility addressing an authentic student problem. By combining a lightweight Flask backend with engineered prompt templates, unstructured lecture notes are reliably transformed into actionable revision study guides.

Role of AI: Generative AI functioned both as the runtime processing brain (via Gemini models) and as a pair-programming collaborator across the development lifecycle—accelerating prototyping, generating test harnesses, and resolving deployment bottlenecks.

Future Scope: Planned future enhancements include: (1) Export functionality enabling one-click download of generated study sheets as Markdown or styled PDFs, (2) Audio lecture transcription to summarize classroom recordings directly, and (3) An interactive quiz mode converting extracted definitions and exam questions into self-grading digital flashcards.